

Rough VS Multifractal Hypothesis Testing

Central Limit Theorem Illustration and $H = 0$ vs. $H \neq 0$ Hypothesis Testing

Technical Documentation

Othmane Zarhali

August 13, 2026

Abstract

This document provides a complete mathematical and software reference for the **rough-vs-multifractal-hypothesis-testing** repository, which implements (i) a Monte Carlo illustration of the Central Limit Theorem, and (ii) a statistical hypothesis-testing pipeline distinguishing a *rough* (fractional, Hurst exponent $H > 0$) log-volatility model from its *multifractal* / log-correlated boundary case $H = 0$. It covers six scripts in full: `clt_illustration.py`, the standalone CLT demonstration; `reference_simulator.py`, the self-contained FFT/circulant-embedding field simulator and independent-fields diagnostics; `clt_check.py`, `regularized_indicator.py`, `hypothesis_test_regularized.py`, and `hypothesis_test_logM.py`, the CLT-check and hypothesis-test scripts; and `compute_one_point_logM.py`, the batch power-curve driver. Every function is documented with its exact mathematical definition, the corresponding implementation, and any known caveats. Two dependencies referenced by the hypothesis-test scripts but not included in the supplied source, `core_simulation.py` and `efficient_field_sim.py`, are documented as interface stubs in their own sections.

Contents

1	Mathematical Background	3
1.1	Driving process covariance	3
1.2	Block statistics	3
1.3	Log-mass CLT and asymptotic variance	3
1.4	Normalized partial sum (CLT illustration)	4
2	Module: <code>clt_illustration.py</code> — CLT Illustration	5
2.1	<code>simulate_normalized_sum(n, n_mc, rng)</code>	5
2.2	<code>plot_clt_grid(n_values, n_mc, seed)</code>	5
3	Module: <code>reference_simulator.py</code> — Reference Field Simulator	6
3.1	<code>sfbm_om_corr(size, dt, T, lambda2, H)</code>	6
3.2	<code>gaussprocess(covariance, size)</code>	6
3.3	<code>genlogVol_independent(size, subsample, T, lambda2, H, M, seed_zz2, seed_lamOmega)</code>	6
3.4	<code>make_figure_std_vs_lambda(lambda_sq_list, n_reps)</code>	7
3.5	<code>make_figure_histograms(lambda_sq_list)</code>	7
3.6	<code>make_figure_qq(lambda_sq_list)</code>	7

4	Module: core_simulation.py — External Dependency (Interface Stub)	8
4.1	build_chol(n , Δ , T , H , λ , δ_{fine}) $\rightarrow (L, K)$	8
4.2	simulate_M($L, K, n, \Delta, \delta_{\text{fine}}, \sigma^2, M_{\text{reps}}, \text{seed}$) $\rightarrow M$	8
4.3	Vn_formula(H, L_n, T) $\rightarrow \text{float}$	8
5	Module: efficient_field_sim.py — External Dependency (Interface Stub)	9
5.1	simulate_Y_batch($n, \Delta, T, H, \lambda, n_{\text{reps}}, \text{pts_per_window}, \sigma^2, \text{seed}, \text{chunk_size}$) $\rightarrow Y$	9
6	Hypothesis Tests for $H = 0$ vs. $H \neq 0$	10
6.1	Module: clt_check.py	10
6.2	Module: regularized_indicator.py	10
6.3	Module: hypothesis_test_regularized.py — statistic T_n	10
6.4	Module: hypothesis_test_logM.py — statistic Z_n	11
6.5	Module: compute_one_point_logM.py	11
7	Script Reference Summary	13
8	Default Parameters and Import	13

1 Mathematical Background

The hypothesis-testing pipeline studies a continuous-time log-volatility field ω_t driven by a centred, stationary Gaussian process. The model is parametrised by three quantities:

- $H \in [0, \frac{1}{2})$ — the **Hurst exponent**, controlling the roughness of the volatility path ($H = 0$ is the logarithmic / multifractal boundary case);
- $\lambda^2 > 0$ — the **volatility-of-volatility** (intermittency) coefficient;
- $T > 0$ — the **integral (decorrelation) scale**, beyond which the covariance of ω_t vanishes.

1.1 Driving process covariance

The log-volatility field ω_t is a centred stationary Gaussian process with autocovariance

$$c_H(x) = \begin{cases} \lambda^2 \log \frac{T}{x}, & H = 0, 0 < x < T, \\ \frac{\lambda^2}{2H(1-2H)} (T^{2H} - x^{2H}), & H \neq 0, 0 < x < T, \\ 0, & x \geq T. \end{cases} \quad (1)$$

Note

$H = 0$ as the multifractal boundary. The $H = 0$ branch of Eq. (1) is the $H \rightarrow 0$ limit of the fractional branch and coincides with the covariance of a *log-correlated* (multifractal) random field, $c_0(x) \sim \lambda^2 \log(1/x)$ as $x \rightarrow 0$. This is exactly the null hypothesis $H_0 : H = 0$ tested against $H_1 : H \neq 0$ throughout Section 6.

1.2 Block statistics

Partitioning the time axis into n windows of length Δ , define on window j :

$$M_j := \int_{\text{window } j} e^{2\omega(t)} dt \quad (\text{integrated-variance proxy}), \quad (2)$$

$$\Omega_j := \int_{\text{window } j} (\omega(t) - \bar{\omega}) dt \quad (\text{raw centred block integral of the field}). \quad (3)$$

The re-centred log-mass $zz2 := \log M - \overline{\log M}$ and the block integral Ω are the two random variables compared in Section 3.

1.3 Log-mass CLT and asymptotic variance

Writing $\Sigma_n := \sum_{j=1}^n (\log M_j - \mathbb{E}[\log M_j])$, every hypothesis test in Section 6 rests on

$$\Sigma_n / \lambda \xrightarrow{d} \mathcal{N}(0, V_n(H)), \quad (4)$$

for a closed-form asymptotic variance $V_n(H)$ depending on the window length $L_n = n\Delta$ and memory scale T (Section 4, `Vn_formula`).

1.4 Normalized partial sum (CLT illustration)

Independently of the field model, `clt_illustration.py` (Section 2) illustrates the classical Lindeberg–Lévy CLT: for $X_1, \dots, X_n$ i.i.d. $\sim \text{Uniform}(-\sqrt{3}, \sqrt{3})$ (mean 0, variance 1),

$$Z_n := \frac{1}{\sqrt{n}} \sum_{i=1}^n X_i \xrightarrow{d} \mathcal{N}(0, 1) \quad \text{as } n \rightarrow \infty. \quad (5)$$

2 Module: `clt_illustration.py` — CLT Illustration

Standalone Monte Carlo illustration of Eq. (5); no dependency on the log-volatility field of Section 1.

2.1 `simulate_normalized_sum(n, n_mc, rng)`

Draws `n_mc` i.i.d. replications of Z_n (Eq. (5)) for a given n , by sampling $X \sim \text{Uniform}(-\sqrt{3}, \sqrt{3})$ of shape `(n_mc, n)` and returning `X.sum(axis=1)/sqrt(n)`. Returns an array of shape `(n_mc,)`.

2.2 `plot_clt_grid(n_values, n_mc, seed)`

Algorithm.

1. For each $n \in \text{n_values}$ (default $\{1, 2, 5, 10, 20, 50, 100\}$), call `simulate_normalized_sum` with $N_{\text{MC}} = \text{n_mc}$ (default 50,000).
2. Plot the empirical histogram of Z_n against the $\mathcal{N}(0, 1)$ density on a fixed 2×4 subplot grid.
3. Remove any unused trailing panel(s) if `n_values` has fewer than 8 entries.

Saves `clt_illustration.{pdf,png}`.

3 Module: `reference_simulator.py` — Reference Field Simulator

Self-contained FFT/circulant-embedding simulator of the field of Eq. (1) and its independent-fields null-hypothesis diagnostics. No external dependency.

3.1 `sfbm_om_corr(size, dt, T, lambda2, H)`

Returns (m, cc) : the autocovariance sequence $cc_k = c_H(k \cdot dt)$, $k = 0, \dots, \text{size} - 1$ (Eq. (1)), together with the mean shift $m = -cc_0$ so that the simulated field has the correct stationary mean.

Parameter	Default	Description
<code>size</code>	required	Number of covariance lags to generate.
<code>dt</code>	0.1	Time-grid spacing.
<code>T</code>	200	Memory-length / cutoff parameter.
<code>lambda2</code>	0.02	Volatility-of-volatility (variance scale).
<code>H</code>	0	Roughness exponent; $H=0$ selects the logarithmic kernel, $H \neq 0$ the fractional kernel.

3.2 `gaussprocess(covariance, size)`

Simulates a stationary centred Gaussian process of length `size` with the given autocovariance, via Wood–Chan circulant embedding.

Algorithm.

1. Choose $m = 2^{\lceil \log_2(\text{size}) \rceil}$, block length $M = 2m$.
2. Zero-pad the covariance to length $m + 1$; build the symmetric circulant row $r = [c_0, \dots, c_m, c_{m-1}, \dots, c_1]$ (length M).
3. Compute eigenvalues $\Lambda_k = \text{Re}(\text{FFT}(r))_k$.
4. Draw i.i.d. $u_k, v_k \sim \mathcal{N}(0, 1)$; form $Z_k = \sqrt{\Lambda_k + 0i} (u_k + iv_k) / \sqrt{2}$.
5. Return $x = \text{Re}(\text{IFFT}(Z))[: \text{size}] \cdot \sqrt{2M}$.

Note

Small negative eigenvalues caused by floating-point rounding are absorbed harmlessly by the complex square root $\sqrt{\Lambda_k + 0i}$: the resulting imaginary contribution is discarded by the final $\text{Re}(\cdot)$.

3.3 `genlogVol_independent(size, subsample, T, lambda2, H, M, seed_zz2, seed_lamOmega)`

Draws `zz2` and Ω (`lamOmega`) from **two independent** field realisations sharing the same covariance law (Eq. (1)) — the null-hypothesis benchmark used by all three figures below.

Parameter	Default	Description
size	4096	Number of output blocks.
subsample	8	Fine-grid points per unit time.
T	4096	Memory-length parameter.
lambda2	0.025	Volatility-of-volatility.
H	0	Roughness exponent.
M	32	Fine-grid points aggregated per output block.
seed_zz2, seed_lamOmega	None	Independent RNG seeds for each field draw.

Algorithm.

1. Build (m, corr) once via the internal helper `_corr_and_mean`, shared by both draws below.
2. **Field #1** (seeded by `seed_zz2`): draw $\omega^{(1)}$ via `gaussprocess`; reshape into n blocks of $M \cdot \text{subsample}$ fine points; compute $\log(\sum_i e^{2\omega_i^{(1)}}) + \log(dt)$ per block via `scipy.special.logsumexp` (numerically stable); re-centre to get `zz2`.
3. **Field #2** (seeded by `seed_lamOmega`, independent draw, same covariance law): draw $\omega^{(2)}$; per block, compute $\Omega = \sum_i (\omega_i^{(2)} - \overline{\omega^{(2)}}) dt$.

Returns. `(zz2, lamOmega)`, each of shape `(n_blocks,)`.

3.4 make_figure_std_vs_lambda(lambda_sq_list, n_reps)

Let $R := zz2 - \frac{2}{M}\Omega$. If `zz2` and Ω are generated from independent field realisations, then as $\lambda^2 \rightarrow \infty$, $\text{std}(R) \sim C \cdot \lambda$, i.e. a log-log regression of $\text{std}(R)$ on $\lambda = \sqrt{\lambda^2}$ has slope ≈ 1 . For each λ^2 in `lambda_sq_list` (default $\{10^{16}, 10^2, 10^{-2}\}$), runs `n_reps` (default 8) independent repetitions of `genlogVol_independent`, computes $\text{std}(R)$ per repetition, and fits the log-log power law by weighted least squares (weights $\propto 1/\text{SEM}^2$). Prints the per- λ^2 table and the fitted slope $\pm$ its standard error, and saves `std_vs_lambda.{pdf,png}`.

3.5 make_figure_histograms(lambda_sq_list)

For each λ^2 in `lambda_sq_list` (default $\{10^{16}, 10^2, 10^{-2}, 10^{-4}\}$), overlays the empirical histograms of `zz2` and of the scaled Ω , reporting $\text{std}(R)$ and $\text{corr}(zz2, \frac{2}{M}\Omega)$ in each panel title. Saves `histogram_comparison.{pdf,png}`.

3.6 make_figure_qq(lambda_sq_list)

QQ-plots of the empirical quantiles of the scaled Ω against those of `zz2`, one panel per λ^2 (default $\{10^{16}, 10^2, 10^{-2}, 10^{-4}\}$), with the 45-degree reference line. Saves `qq_plots.{pdf,png}`.

4 Module: `core_simulation.py` — External Dependency (Interface Stub)

Known issue

External dependency, not included in the supplied source. The three functions below were imported by `clt_check.py`, `regularized_indicator.py`, `hypothesis_test_regularized.py`, and `hypothesis_test_logM.py`, but never defined in the original material this repository was built from. Their signatures and roles below are reconstructed purely from their call sites; the shipped module raises `NotImplementedError` in each body rather than guessing at the Cholesky-simulation internals.

4.1 `build_chol(n, Delta, T, H, lam, delta_fine) → (L, K)`

Builds a Cholesky factor L (and companion array K) of the covariance of a fine-grid (spacing `delta_fine`) Gaussian log-volatility path over n windows of length `Delta`, memory scale T , Hurst exponent H , and volatility-of-volatility `lam` (playing the role of $\lambda = \sqrt{\lambda^2}$).

4.2 `simulate_M(L, K, n, Delta, delta_fine, sigma2, M_reps, seed) → M`

Given the factors from `build_chol`, simulates `M_reps` independent realisations of the windowed integrated-variance proxy M_j , $j = 1, \dots, n$ (analogous to e^{zz2} in Section 3), with base variance `sigma2` and RNG seed `seed`. Returns an array of shape (n, M_reps) , consistent with call sites such as `Y = np.log(M/Delta)`.

4.3 `Vn_formula(H, Ln, T) → float`

Closed-form asymptotic variance $V_n(H)$ of Σ_n/λ (Eq. (4)) over a window of total length $L_n = n\Delta$ and memory parameter T , used both to normalise the test statistics T_n , Z_n (Section 6) and to compute the theoretical power curve.

5 Module: `efficient_field_sim.py` — External Dependency (Interface Stub)

Known issue

Same status as Section 4: referenced by `compute_one_point_logM.py` but not defined in the supplied source.

5.1 `simulate_Y_batch(n, Delta, T, H, lam, n_reps, pts_per_window, sigma2, seed, chunk_size) → Y`

Memory-efficient (chunked) counterpart to `simulate_M + log(M/Δ)`: simulates `n_reps` replications of the centred log-mass increments $Y_j = \log(M_j/\Delta)$ over `n` windows, at Hurst exponent `H` and volatility-of-volatility `lam`, using `pts_per_window` fine-grid points per window, processed in batches of `chunk_size` to bound peak memory (needed for the large- T regime, e.g. $T = 20,000$, of `compute_one_point_logM.py`). Returns an array of shape `(n_reps, n)`.

6 Hypothesis Tests for $H = 0$ vs. $H \neq 0$

6.1 Module: `clt_check.py`

CLT check on the log-mass statistic at $\lambda^2 = 10^{-2}$, $n = 1$, $T = 1.05$ deliberately close to $\Delta = 1$ (weak within-window correlation, $\rho = 1 - (\Delta/T)^{2H} \approx 0.019$ at $H = 0.2$), included for completeness even though at this small λ the Gaussian fit is expected to hold regardless of T .

Algorithm.

1. Simulate M via `build_chol` + `simulate_M` at $H = 0.2$, $\lambda = 0.1$, $M_{\text{reps}} = 30,000$.
2. $Y = \log(M/\Delta)$, $\Sigma_n = \sum_j Y_j$, $W := (\Sigma_n - \overline{\Sigma_n})/\lambda$.
3. Fit $\mathcal{N}(\overline{W}, \text{std}(W)^2)$; compute skewness, kurtosis, and a Kolmogorov–Smirnov p -value against the standardised normal.

Outputs. Figure A — histogram of W vs. fitted Gaussian (`clt_hist_lambda2_1em2.{pdf,png}`); Figure B — QQ-plot of standardised W (`clt_qq_lambda2_1em2.{pdf,png}`).

6.2 Module: `regularized_indicator.py`

Fix a threshold a and bandwidth $\varepsilon > 0$. With $Y_j := \log M_j - \mathbb{E}[\log M_j]$,

$$\hat{p}_n^\varepsilon := \frac{1}{n} \sum_{j=1}^n \Phi\left(\frac{Y_j - a}{\varepsilon}\right) \xrightarrow{\varepsilon \rightarrow 0} \hat{p}_n := \frac{1}{n} \sum_{j=1}^n \mathbf{1}\{Y_j \geq a\} \quad (\text{pointwise}). \quad (6)$$

At $n = 5$, $T = 8$, $\Delta = 1$, $H = 0.2$, $\lambda = 0.1$, $a = 0.1$, for $\varepsilon \in \{1, 0.1, 0.01, 10^{-4}\}$: overlays the histogram of $\hat{p}_n^\varepsilon$ against the (discrete) pmf of $\hat{p}_n$ (`phat_vs_hard_n5.{pdf,png}`); reports mean, variance, and exact-match fraction at each ε ; and, at the smallest $\varepsilon = 10^{-4}$, produces a nearest-point pmf comparison table between the two statistics.

6.3 Module: `hypothesis_test_regularized.py` — statistic T_n

With $c_\varepsilon := \varphi(a/\varepsilon)/\varepsilon$ (φ the standard normal density),

$$T_n := \frac{n(\hat{p}_n^\varepsilon - \Phi(-a/\varepsilon))}{\lambda c_\varepsilon \sqrt{V_n(0)}}. \quad (7)$$

Reject H_0 at level α when $|T_n| > z_{1-\alpha/2}$. Under H_0 , $T_n \xrightarrow{d} \mathcal{N}(0, 1)$ (asymptotic size α). Under a true H , with $R(H) := \sqrt{V_n(H)/V_n(0)}$, the asymptotic power is $\pi(H) = 1 - \Phi(z_{1-\alpha/2}/R(H)) + \Phi(-z_{1-\alpha/2}/R(H))$.

Parameter	Value	Notes
n, Δ, T	5, 1, 8	
λ	0.05	Chosen for size closest to target among $\{0.1, 0.05, 0.03\}$.
a, ε	0.1, 0.3	$\varepsilon \gg \varepsilon^* = \sqrt{a\lambda}$ (delta-method-valid regime).
α	0.05	
H_{proxy}	0.01	Numerically safe stand-in for simulating $H = 0$ (the field variance $\nu^2 = \lambda^2/(H(1 - 2H))$ diverges at literal $H = 0$).
H grid	$\{0.05, \dots, 0.45\}$	Step 0.05, for the power curve.

Outputs. Console: $V_n(0)$, c_ε , z_{crit} , empirical size at $H = 0$, per- H empirical/theoretical power. Figure: `test_size_power.{pdf,png}`.

6.4 Module: `hypothesis_test_logM.py` — statistic Z_n

$$Z_n := \frac{\Sigma_n}{\lambda \sqrt{V_n(0)}}, \quad \Sigma_n = \sum_{j=1}^n \log(M_j/\Delta). \quad (8)$$

Reject H_0 when $|Z_n| > z_{1-\alpha/2}$. Z_n and T_n share the same asymptotic ($n \rightarrow \infty$) size and power in the delta-method-valid regime, but Z_n requires no choice of threshold a or bandwidth ε and exhibits noticeably better finite-sample power.

Note

Choice of H_{proxy} . $H_{\text{proxy}} = 0.01$ was tried first and found to induce a genuine numerical-conditioning issue in the Cholesky simulator near the $H \rightarrow 0$ singular boundary: the linear Gaussian part alone (bypassing the log-mass nonlinearity) showed an 8.8% variance inflation relative to the theoretical $V_n(H)$, while $V_n(H)$ itself was independently confirmed correct via direct double-integration of the covariance function. $H_{\text{proxy}} = 0.05$ shows excellent agreement (ratio 0.99–1.01) and is used instead.

Parameters otherwise as in `hypothesis_test_regularized.py` except $\lambda = 0.1$ (size already excellent here) and $H_{\text{proxy}} = 0.05$. **Outputs.** Console analogue of the above; figure `logM_test_size_power.{pdf,png}`.

6.5 Module: `compute_one_point_logM.py`

Computes the empirical power $\hat{\pi}(H, n)$ of the Z_n test for a *single* (H, n) pair via `simulate_Y_batch` (chunked, for the large- $T = 20,000$ regime), and appends `{"H": H, "n": n, "power": power}` as one JSON line to `power_results_logM.jsonl`. Intended to be run as a fresh process per grid point (e.g. from a job array).

Argument	Type	Description
<code>H</code>	float, positional	Alternative Hurst exponent to evaluate power at.
<code>n</code>	int, positional	Number of windows.

Usage. `python compute_one_point_logM.py <H> <n>`. **Fixed parameters.** $\Delta = 1$, $T =$

20,000, $\lambda = 0.1$, $\alpha = 0.05$, $H_{\text{proxy}} = 0.05$, $\text{pts_per_window} = 50$, $n_{\text{calib}} = 500$, $n_{\text{reps}} = 300$, $\text{chunk_size} = 25$.

7 Script Reference Summary

Script	Test statistic / output	Primary use
<code>reference_simulator.py</code>	$\text{std}(R)$, histograms, QQ	Validates independent-fields null benchmark
<code>clt_check.py</code>	$W = \Sigma_n / \lambda$	CLT sanity check on the log-mass statistic
<code>regularized_indicator.py</code>	$\hat{p}_n^\varepsilon$ vs. $\hat{p}_n$	Regularization-bandwidth diagnostic
<code>hypothesis_test_regularized.py</code>	T_n	$H = 0$ vs. $H \neq 0$ test, Theorem 24
<code>hypothesis_test_logM.py</code>	Z_n	$H = 0$ vs. $H \neq 0$ test, direct log-mass
<code>compute_one_point_logM.py</code>	$\hat{\pi}(H, n)$	Batch power curve construction

8 Default Parameters and Import

Parameter	Default	Notes
<code>subsample</code>	8	Fine-grid oversampling in <code>reference_simulator.py</code> .
<code>M</code>	32	Fine-grid points per output block (<code>reference_simulator.py</code>).
α	0.05	Nominal test level throughout Section 6.
H_{proxy}	0.01 or 0.05	See per-script note; 0.05 preferred where finite-sample conditioning was checked.

Import.

```
from reference_simulator import genlogVol_independent
from core_simulation import build_chol, simulate_M, Vn_formula
from efficient_field_sim import simulate_Y_batch
```